Ders 21 Çalışma Özeti

Ders 21 Çalışma Özeti

Genel Konular

- Dersin tarihi 16 Aralık 2020 Çarşamba, dönemin 11. haftası. Ders Zoom üzerinden yapıldı.
  - Kaydın 00:59:50 ile 01:12:00 arası ara vermedir.
- Duyuru: ikinci vize yapılmayacak.
- Üçüncü ödevle (yazım denetleyici) ilgili öğrenci soruları.
- En uzun ortak alt dizinin (LCS) tamamlanması
  - Yön matrisi B: ekle (add), x'te atla (skip x), y'de atla (skip y).
  - LCS'yi geri izleyerek yazdıran kod.
- Graflara giriş ve tekrar
  - Kullanım alanları: en kısa yol, torrent, web tarama (crawling), topolojik sıralama, sosyal ağlar.
  - Temel kavramlar: düğüm (vertex), kenar (edge), yönlü (directed) ve yönsüz (undirected) graf.
  - Saklama: komşuluk matrisi (adjacency matrix) ve komşuluk listesi (adjacency list); yoğun (dense) ve seyrek (sparse) graf.
  - Tam graf (complete graph), ağırlıklı graf (weighted graph), bağlılık (connectivity), döngüsüzlük (acyclicity), bağlı bileşen (connected component), orman (forest).
- Derinlik öncelikli arama (depth-first search, DFS) ve genişlik öncelikli arama (breadth-first search, BFS)
  - Sözde kodlar, kullandıkları veri yapıları, karşılaştırma ve karmaşıklık.
  - Kullanım örnekleri: labirent, en yakın noktalar, döngü bulma.
- Topolojik sıralamaya (topological sorting) giriş: ders ön koşulları örneği. Konu ders 22'de tamamlanır.

Hocanın Özellikle Vurguladığı Kısımlar

- **İkinci vize yapılmayacak.** İkinci vize notu ödevler ve dönem projesiyle oluşturulacak.
- Ödevde yazılanlar zorunludur. Fazlası (örneğin sözlüğe ekleme özelliği) faydalıysa olumlu değerlendirilir, ama yapılmadı diye puan kırılmaz.
  - Hoca, pandemi döneminde ödevleri çok genişletmemeye özen gösterdiğini söyledi.
- LCS'nin uzunluğunu bulmak yetmez. Hangi karakterlerden oluştuğunu bulmak için ikinci bir matris ve ek kod gerekir.
- Graflarda hangi saklama yapısının seçileceği kenar ve düğüm sayısına birlikte bağlıdır.
- Karmaşıklığı belirleyen parametreler yalnız düğüm ve kenar sayısı değildir. Grafın saklandığı yapı da karmaşıklığı doğrudan etkiler.
- DFS ve BFS bütün grafı gezdiği için en iyi ve en kötü durum aynıdır. Bu yüzden karmaşıklık Θ ile verilir.
- DFS ve BFS'nin işleyişi sınavda karşınıza çıkabilir. Hazırlıklı olun.
- Hoca, derse girip takip etmeyen ve söz verildiğinde cevap vermeyen öğrencilere sitem etti. Yanlış da olsa cevap vermeyi denemenizi istedi.

Kısa Tekrar Notları

- LCS yön matrisi:
  - $x_i = y_j$ ise $B[i, j] =$ add (örneğin 0).

  - $C[i-1,j] \geq C[i,j-1]$ ise $B[i,j] =$ skip x (örneğin 1): x_i LCS'de yok.
  - Aksi hâlde $B[i,j] =$ skip y (örneğin 2): y_j LCS'de yok.
- Geri izleme sağ alt köşeden başlar, $i \neq 0$ ve $j \neq 0$ olduğu sürece sürer.
- Graf saklama:
  - Komşuluk matrisi: yer $\Theta(|V|^2)$. Yoğun graflar için uygun.
  - Komşuluk listesi: yer $\Theta(|V| + |E|)$. Seyrek graflar için uygun.
  - Ağırlıklı grafta kenar olmayan yerler matriste ∞ ile gösterilir. Listede komşu ile birlikte ağırlık da saklanır.
- DFS yığın (stack), BFS kuyruk (queue) kullanır.
- DFS'te iki sıra vardır: ziyaret (push) sırası ve çıkarma (pop) sırası. BFS'te tek sıra vardır.
- DFS ormanında ağaç (tree) ve geri (back) kenarlar, BFS ormanında ağaç ve çapraz (cross) kenarlar oluşur.
- Karmaşıklık (DFS ve BFS için aynı): matriste $\Theta(|V|^2)$, listede $\Theta(|V| + |E|)$.
- Bağlı bileşen sayısı: dıştaki döngü DFS'i her yeniden başlattığında bir bileşen bitmiş olur.

Detaylı Açıklamalar

Üçüncü ödevle ilgili sorular

Üçüncü ödev, düzenleme uzaklığı ve hash tablolarıyla çalışan bir yazım denetleyicidir.

- Kullanıcıya önerilen kelimeler dışında bir kelime seçtirilmez. Seçim önerilen listeden yapılır.
  - Seçimi numarayla da kelimeyi yazdırarak da alabilirsiniz. Kelime yazdırılıyorsa yanlış girişi mutlaka kontrol edin.
- Bir hatalı kelime daha önce düzeltilip hatalı kelime tablosuna eklendiyse, tekrar geldiğinde onun kayıtlı doğru karşılığı önerilir. Tekrar giriş almak gerekmez.
  - Birden fazla karşılık tutup seçenek olarak sunmak da doğru bir uygulama olabilir.
- Düzenleme uzaklığı sınırı (örneğin 2) içinde yakın kelime yoksa kelime değiştirilmeden bırakılır.
  - Sınırı değişken olarak tutarsanız 3'e de çıkarabilirsiniz.
- Hatalı kelimeler ve doğru karşılıkları anahtar-değer (key-value) mantığıyla saklanmalıdır: bir hatalı kelimeye bir doğru karşılık.

Değerlendirme (yaklaşık)

- Dönem içi %60, final %40.
- Hoca, diğer grubun hocasıyla henüz kesinleştirmediklerini söyleyerek yaklaşık bir dağılım verdi:
  - Yapılan sınav yaklaşık %20.
  - Dönem projesi yaklaşık %20.
  - Ödevler yaklaşık %20.
  - Proje ile ödevler arasında oran kayabilir.
- Bir dönem projesi kesin verilecek. Bir ödev daha verilmesi de olası.

LCS'nin geri izlenmesi

İlk aşamada C matrisi doldurulurken bir de B yön matrisi doldurulur:

```
if x[i] == y[j]:
    C[i,j] <- C[i-1,j-1] + 1
    B[i,j] <- add
else if C[i-1,j] >= C[i,j-1]:
    C[i,j] <- C[i-1,j]
    B[i,j] <- skip x        # x[i] LCS'de yok
else:
    C[i,j] <- C[i,j-1]
    B[i,j] <- skip y        # y[j] LCS'de yok
```

İkinci aşamada sağ alt köşeden geri gidilir:

```
i <- m ; j <- n
while i != 0 and j != 0:
    switch B[i,j]:
        case add:    LCS'nin başına x[i] ekle ; i <- i-1 ; j <- j-1 ; break
        case skip x: i <- i-1 ; break
        case skip y: j <- j-1 ; break
```

- **add:** Son eşleşen karakter budur. LCS'nin başına eklenir. Çaprazdan gelindiği için i ve j birlikte azalır.
- **skip x:** Yukarıdan gelinmiştir. Bir üst satıra çıkılır.
- **skip y:** Soldan gelinmiştir. Bir sol sütuna geçilir.
- Geçen haftaki BACDB ve BDCB örneğinde iki LCS (BCB ve BDB) vardı. Hangisinin bulunacağı, eşitlik durumunda hangi yönün seçildiğine bağlıdır.

Graflarla çözülen problemler ve temel kavramlar

- **En kısa yol:** Dijkstra algoritması; ağırlıksız graflarda BFS.
- **Ağaç:** Grafın özel bir hâlidir. Grafın ağaç olabilmesi için içinde döngü (cycle) olmamalıdır.
- **Diğer kullanımlar:** Torrent'te parçaların hangi düğümden geleceği, web tarayıcıların (crawler) sayfaları dolaşması, topolojik sıralama (ders ön koşulları, yazılım kurulum bağımlılıkları), sosyal ağlar.
- **Saklama yapıları:**
  - Kenar sayısı düğüm sayısına göre çoksa graf yoğundur, komşuluk matrisi tercih edilir.
  - Kenar sayısı azsa graf seyrektir, komşuluk listesi tercih edilir. Örneğin 5 düğümlü bir grafta yalnız 2 kenar varsa matrisin geri kalanı boş (0) kalır.
- **Bağlılık:** Her düğüm çiftinin arasında bir yol varsa graf bağlıdır. Örnekte A, B, C, D, E bir bileşen; F, G ikinci bileşendir.
- **Orman:** Birbirine bağlı olmayan birden fazla ağaçtan oluşan yapıdır.
  - Hoca, karar ağaçlarını (decision tree) kullanan rastgele orman (random forest) sınıflandırma algoritmasından da kısaca söz etti. Birden fazla bağımsız ağacın kararları birleştirilerek ortak bir karar verilir.

DFS

Hocanın yazdığı sözde kod:

```
DFS(G):
    mark each vertex in V with 0      # ziyaret edilmedi
    count <- 0
    for each vertex v in V do
        if v is marked with 0
            dfs(v)

dfs(v):
    count <- count + 1 ; mark v with count
    for each vertex w in V adjacent to v do
        if w is marked with 0
            dfs(w)
```

- DFS, özyinelemeli (recursive) yaklaşımla yazılır.
- Özyineleme, bir düğümün ziyaret edilmemiş komşusu kalmayınca biter. Algoritmanın tamamı ise bütün düğümler ziyaret edilince biter.
- Yığına sırayla ziyaret edilen düğümler eklenir (push). Bir düğümün ziyaret edilmemiş komşusu kalmayınca yığından çıkarılır (pop) ve bir önceki düğüme dönülür.
- **Örnek (A-B, B-C, B-D, B-E, D-E, F-G):** A'dan başlanır. Eşitlikte alfabetik sıra seçilir.
  - A, B, C ziyaret edilir. C'nin gidecek yeri kalmayınca çıkarılır, B'ye dönülür.
  - B'den D'ye, D'den E'ye gidilir. Sonra E, D, B, A sırayla çıkarılır.
  - Yığın boşalınca ilk bağlı bileşen biter. Dıştaki döngü ziyaret edilmemiş F'yi bulur ve yeni bir DFS başlatır.
- **Bağlı bileşen sayısı:** Dıştaki döngüde DFS her çağrıldığında ve döndüğünde bir bileşen bitmiş olur. Sayaç buraya konur.

BFS

```
BFS(G):
    mark each vertex in V with 0
    count <- 0
    for each vertex v in V do
        if v is marked with 0
            bfs(v)

bfs(v):
    count <- count + 1 ; mark v with count
    initialize a queue with v
    while the queue is not empty do
        for each vertex w in V adjacent to the front vertex do
            if w is marked with 0
                count <- count + 1 ; mark w with count
                add w to the queue
        remove the front vertex from the queue
```

- Bir düğümün bütün komşuları kuyruğa eklenir. Sonra öndeki düğüm kuyruktan çıkarılır ve sıradaki düğümün komşularına geçilir.
- Örnekte A'dan başlanınca önce B, C, D kuyruğa girer. A çıkarılır. Sonra B'nin ziyaret edilmemiş komşusu E eklenir.

DFS ve BFS karşılaştırması

	DFS	BFS
Veri yapısı	Yığın	Kuyruk
Düğüm sıralaması sayısı	2 (push ve pop sırası)	1
Kenar türleri	Ağaç ve geri kenarlar	Ağaç ve çapraz kenarlar
Karmaşıklık (komşuluk matrisi)	$\Theta(\|V\|^2)$	$\Theta(\|V\|^2)$
Karmaşıklık (komşuluk listesi)	$\Theta(\|V\| + \|E\|)$	$\Theta(\|V\| + \|E\|)$

- Matriste bir düğümün komşularını bulmak için satırın tamamı taranır. Her satır için yapıldığından $|V|^2$ çıkar.
- Listede her düğüm ve her kenar bir kez işlenir. Sonuç $|V| + |E|$.
- Bir öğrenci, karmaşıklığın en kötü durum için mi verildiğini sordu. Hoca, tüm graf gezildiği için alt ve üst sınırın aynı olduğunu, bu yüzden Θ kullanıldığını söyledi. Belirli bir problem (örneğin döngü arama) için en iyi ve en kötü durum farklı olabilir.

Kullanım örnekleri

- **Labirentten çıkış:** DFS. Gidilebilecek yere kadar ilerlenir, çıkmaza girince bir adım geri dönülür.
- **GPS'te en yakın noktalar:** Derste BFS mantığıyla anlatıldı: önce en yakın komşu noktalara bakılır.
- **Döngü bulma:** İkisiyle de yapılabilir. Hoca, verimliliklerinin yaklaşık aynı olduğunu söyledi.
- **Bağlılık kontrolü:** İkisiyle de yapılabilir.

Topolojik sıralamaya giriş

- Bir adım tamamlanmadan diğerine geçilemeyen problemler için kullanılır. Örneğin ders ön koşulları.
- Yönlü graf ile modellenir. Örnek: C1 → C3, C2 → C3, C3 → C4, C3 → C5, C4 → C5.
- Graf döngüsüzdür (acyclic). C3, C4, C5 arasında döngü varmış gibi görünse de C5'ten C3'e kenar yoktur.

- DFS ile gezilir (C1'den başlayarak, eşitlikte küçük numara):
  - Yığına C1, C3, C4, C5 eklenir. Sonra C5, C4, C3, C1 çıkarılır.
  - Yığın boşalınca C2'den yeni DFS başlar. C2 eklenir ve çıkarılır.
  - Çıkarma (pop) sırası: C5, C4, C3, C1, C2.
- Uygun ders sıraları: C1, C2, C3, C4, C5 ya da C2, C1, C3, C4, C5.
- Ders, "DFS sırasında hangi bilgiyle sıralama yazdırılır?" sorusuyla kapandı. Cevap ders 22'de verildi: çıkarma sırasının tersi.

Ders sonrası notlar

- **Topolojik sıralamada ön koşul VE ile bağlıdır, VEYA ile değil.** Derste bir öğrenci, C3'ü almak için C1 ya da C2'den birinin yeterli olduğunu söyledi. Hoca bunu kabul etti. Doğrusu: Öncelik grafında $u \to v$ kenarı "u, v'den önce gelmeli" demektir. C3'e gelen iki kenar varsa C1 ve C2'nin ikisi de C3'ten önce gelmelidir. Hocanın sonradan verdiği iki geçerli sıra da (C1, C2, … ve C2, C1, …) bu anlama uyar. Python ile bütün sıralar denendi: geçerli sıra tam olarak bu ikisidir.
- **Push sırası topolojik sıra değildir.** Derste bir öğrenci push sırasını önerdi. Bu sıra C1, C3, C4, C5, C2'dir ve geçersizdir, çünkü C2, C3'ten sonra gelir. Bir başka öğrencinin önerdiği "pop sırasının tersi" doğrudur: C2, C1, C3, C4, C5.
- **BFS özyinelemeli değildir.** Derste "BFS'ye en uygun yaklaşım özyinelemelidir" dendi. Yukarıdaki $bfs(v)$ fonksiyonu kendini çağırmaz, kuyruk ve `while` döngüsüyle çalışır. Yani BFS yinelemelidir (iterative). Özyineleme DFS'in doğasına uygundur.
- **Tam graf:** Derste "herhangi bir düğüme kenar gitmiyorsa tam değildir" dendi. Bu, düğümün yalıtılmış olmamasıdır. Tam grafta ise her düğüm çifti doğrudan bir kenarla bağlıdır. n düğümlü yönsüz tam grafta $n(n-1)/2$ kenar vardır.
- **Ağırlıklı graf:** Ağırlıklar düğümlere değil kenarlara aittir. Derste bir öğrenci "her düğümün ağırlığı var" dedi.
- **Yoğun graf:** Kenar sayısının düğüm sayısına yakın olması değil, $|V|^2$'ye yakın olması yoğunluk demektir. Kenar sayısı $|V|$ civarındaysa graf seyrektir.
- **Ağaç koşulu:** Yalnız döngüsüz olmak yetmez. Graf aynı zamanda bağlı olmalıdır. Döngüsüz ama bağlı olmayan graf ormandır. n düğümlü ağaçta $n-1$ kenar vardır.
- **GPS:** Yol ağları ağırlıklıdır (mesafe, süre). En kısa yol için BFS değil Dijkstra ya da A* kullanılır. BFS yalnız ağırlıksız graflarda en az kenarlı yolu verir.
- **Döngü bulma:** Yönsüz grafta DFS ve BFS ikisi de kullanılabilir. Yönlü grafta DFS'te geri kenar (back edge) bulunması döngü demektir.
- Ağırlıklı grafta kenar olmayan yer için derste "bazen eksi değer" de dendi. Negatif değer gerçek bir negatif ağırlıkla karışabilir. Bu yüzden genelde ∞ (çok büyük bir sayı) kullanılır.
- Altyazıda "Bread-first", "Brad First" diye geçen terim breadth-first, "Dev-first" diye geçen terim depth-first, "Pistoda" ve "Fisoda" diye geçen terim sözde kod, "asayklik" diye geçen terim acyclic'tir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.